Pontelo: Documentação Técnica

Oct 8, 2026 · @Emerson Mazzeto

1. Visão geral

O Pontelo é uma plataforma web que conecta pessoas em situação de vulnerabilidade a serviços públicos e gratuitos próximos a elas (abrigos, refeitórios, atendimento jurídico, vacinação, CRAS), por meio de um mapa colaborativo e verificado.

> Uma ponte construída com elos porque ninguém precisa caminhar sozinho.

Problema. Muita gente não sabe que esses serviços existem perto de casa. A informação está espalhada em sites de governo, planilhas e redes sociais, quase sempre desatualizada e difícil de achar com um celular simples e pouco pacote de dados.

Solução. Um mapa e uma busca únicos, em linguagem simples, onde qualquer pessoa encontra o serviço certo, vê se ele foi verificado e sabe como chegar. Voluntários e ONGs mantêm os dados atualizados, e moderadores confirmam cada cadastro.

Nome. Pontelo = ponte + elo: a plataforma é a ponte, e cada serviço, voluntário e usuário é um elo.

Perfis de usuário

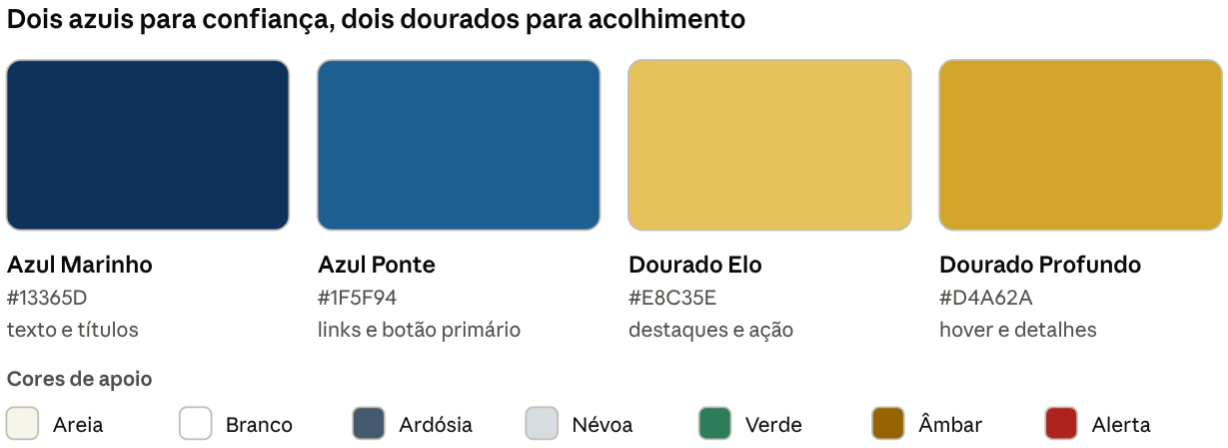
Perfil	O que faz	Precisa de login
Visitante	Busca serviços, vê o mapa, filtra por categoria e distância	Não
Colaborador	Sugere novos serviços, corrige dados, confirma que um serviço ainda funciona	Sim
Organização	Gerencia os serviços que oferece (horários, vagas, contatos)	Sim
Moderador	Verifica sugestões, resolve denúncias, mantém a qualidade dos dados	Sim
Consumidor da API	Pesquisadores e órgãos públicos usam os dados abertos	Chave de API

Objetivos do projeto

- Entregar um sistema completo (front-end, back-end, API REST e banco de dados) que atenda aos requisitos da disciplina.
- Fazer a busca por serviço mais próximo funcionar bem em celulares simples e conexões lentas.
- Manter a confiança nos dados com verificação por moderadores e confirmações da comunidade.
- Abrir os dados por uma API pública e documentada, para que outros projetos possam reutilizá-los.

2. Identidade visual

A paleta do Pontelo tem quatro cores tiradas direto da logo: dois azuis, que passam confiança e estabilidade, e dois dourados, que passam acolhimento e esperança. Os códigos abaixo foram estimados a partir da imagem da logo; confiram com o conta-gotas no arquivo original antes de fixar no projeto.



paleta do Pontelo · 4 cores da marca e 7 de apoio

Cores da marca

Cor	HEX	RGB	Onde aparece na logo	Uso no sistema
Azul Marinho	`#13365D`	19, 54, 93	"Pont" e lado escuro da figura azul	Texto principal, títulos, cabeçalho, rodapé
Azul Ponte	`#1F5F94`	31, 95, 148	Pessoa azul e arco	Links, botões primários, ícones ativos, marcadores do mapa

Cor	HEX	RGB	Onde aparece na logo	Uso no sistema
Dourado Elo	`#E8C35E`	232, 195, 94	Letra "e" e pessoa dourada	Destaques, botão de ação principal (com texto marinho), selos
Dourado Profundo	`#D4A62A`	212, 166, 42	"lo" e lado escuro da figura dourada	Hover e estado pressionado, bordas de destaque, ilustrações

Cores de apoio (neutras e de estado)

Função	Nome	HEX	Observação
Fundo da página	Areia	`#F8F5EE`	Off-white quente, mais suave que branco puro
Superfície (cards, modais)	Branco	`#FFFFFF`	Sobre o fundo Areia
Texto secundário	Ardósia	`#4A5A6E`	Legendas, textos de apoio
Bordas e divisores	Névoa	`#D9DEE5`	Linhas finas
Serviço verificado	Verde Confirmado	`#2E7D5B`	Selo "verificado" nos cards e no mapa
Serviço a confirmar	Âmbar	`#9A6700`	Dados antigos ou ainda sem verificação
Erro e denúncia	Vermelho Alerta	`#B3261E`	Mensagens de erro, ações destrutivas

Contraste (meta: WCAG AA, mínimo 4,5:1 para texto comum)

Combinação	Razão aproximada	Pode usar para texto?
Marinho sobre branco	12,2:1	Sim
Marinho sobre Areia	11,2:1	Sim
Marinho sobre Dourado Elo	7,2:1	Sim (botão de ação)
Azul Ponte sobre branco	6,7:1	Sim (links)
Branco sobre Azul Ponte	6,7:1	Sim (botão primário)

Combinação	Razão aproximada	Pode usar para texto?
Ardósia sobre branco	7,1:1	Sim
Verde Confirmado sobre branco	5,0:1	Sim
Dourado Profundo sobre branco	2,3:1	**Não**, só decoração
Dourado Elo sobre branco	1,7:1	**Não**, só decoração

Regra prática: os dourados nunca servem de cor de texto sobre fundo claro. Em botões dourados, o texto é sempre Azul Marinho. As razões foram calculadas à mão com a fórmula do WCAG; validem as finais no WebAIM Contrast Checker.

Tokens de design (CSS)

```
:root {
  /* marca */
  --pontelo-marinho: #13365D;
  --pontelo-azul: #1F5F94;
  --pontelo-dourado: #E8C35E;
  --pontelo-dourado-escuro: #D4A62A;

  /* apoio */
  --fundo: #F8F5EE;
  --superficie: #FFFFFF;
  --texto: var(--pontelo-marinho);
  --texto-secundario: #4A5A6E;
  --borda: #D9DEE5;

  /* estado */
  --verificado: #2E7D5B;
  --a-confirmar: #9A6700;
  --erro: #B3261E;
}
```

Tipografia

- **Títulos:** Nunito (700 a 800). É uma sans arredondada, parecida com o desenho das letras da logo.
- **Texto corrido:** Inter ou a fonte do sistema, no mínimo 16 px, com altura de linha de 1,5.
- **Desempenho:** hospedar as fontes no próprio site (formato WOFF2, só os pesos usados) para economizar dados do usuário.

Regras de uso da logo

- Usar sobre fundo claro (Areia ou branco). Para fundo escuro, criar uma versão toda branca.
- Manter margem livre ao redor igual à altura da letra "P".
- Não redistorcer, não trocar as cores e não aplicar sombra.
- Gerar a partir da logo atual: versão em SVG, versão só com o símbolo (as duas pessoas e a ponte) para favicon e ícone de app (192 e 512 px) e versão horizontal para o cabeçalho.

3. Requisitos

O MVP (versão mínima para apresentar) cobre a busca, o mapa, o cadastro colaborativo com moderação e a API pública; o que está marcado como desejável entra se sobrar tempo.

Requisitos funcionais

ID	Requisito	Prioridade
RF01	Buscar serviços por categoria, localização e distância	MVP
RF02	Ver os serviços em um mapa, com marcadores por categoria	MVP
RF03	Ver detalhes do serviço: endereço, horários, telefone, documentos exigidos e link "como chegar"	MVP
RF04	Cadastro e login para colaborador, organização e moderador	MVP
RF05	Sugerir um novo serviço	MVP
RF06	Moderador aprova, rejeita ou pede ajustes em sugestões	MVP
RF07	Botão "confirmo que ainda funciona", que atualiza a data da última verificação	MVP
RF08	Denunciar informação incorreta ou serviço que fechou	MVP
RF09	API pública de leitura, documentada, com chave de acesso	MVP

ID	Requisito	Prioridade
RF10	Buscar por CEP ou cidade, convertendo para coordenadas	MVP
RF11	Filtro "aberto agora" para quem precisa de ajuda urgente	Desejável
RF12	Painel da organização para gerenciar seus serviços	Desejável
RF13	Funcionar offline com os últimos resultados pesquisados (PWA)	Desejável
RF14	Painel de estatísticas com regiões mal atendidas por categoria	Desejável

Requisitos não funcionais

Área	Requisito (metas propostas)
Desempenho	Página inicial carrega em até 3 s em conexão 3G simulada; nota Lighthouse de desempenho acima de 90
Acessibilidade	Conformidade com WCAG 2.1 nível AA; navegação completa por teclado; compatível com leitor de tela
Compatibilidade	Funcionar em Android antigo e em navegadores de duas versões atrás; layout pensado primeiro para celular
Segurança	Senhas com hash forte, HTTPS, validação de entrada, limite de requisições por IP
Privacidade	Coletar o mínimo de dados pessoais; buscas anônimas por padrão; adequação à LGPD
Qualidade	Testes automatizados na API; integração contínua rodando testes a cada push
Documentação	API descrita em OpenAPI 3 com interface Swagger
Manutenção	Ambiente local reproduzível com um único comando (Docker Compose)

4. Stack tecnológica

A stack é TypeScript de ponta a ponta (React no front, Node.js no back) com PostgreSQL + PostGIS no banco, tudo gratuito e de código aberto. Escolhi assim porque o grupo não informou preferência; como o contrato da API será descrito em OpenAPI, dá para trocar o back-end (Python, Java) sem refazer o front.

Tecnologias por camada

Camada	Tecnologia	Por que
Front-end	React + TypeScript, empacotado com Vite	Ecossistema grande, build rápido, tipagem reduz bugs
Estilo	Tailwind CSS, com os tokens da paleta	Layout mobile-first e consistência visual com pouco CSS
Mapa	Leaflet + react-leaflet, tiles do OpenStreetMap	Gratuito, leve e sem cartão de crédito, ao contrário do Google Maps
Dados no front	TanStack Query e React Router	Cache de requisições (economiza dados) e rotas
PWA	vite-plugin-pwa (service worker)	Instalável no celular e com cache offline dos últimos resultados
Back-end	Node.js (versão LTS) + Fastify + TypeScript	Rápido, validação por schema nativa, plugin oficial de Swagger
Validação	Zod	Mesmo schema valida entrada da API e tipos do código
Acesso a dados	Prisma ORM, mais SQL puro nas consultas geográficas	Prisma cobre o CRUD; o SQL puro cobre PostGIS, que o ORM não suporta bem
Autenticação	JWT (token de acesso curto + refresh token), senhas com Argon2	Padrão de mercado, sem estado no servidor
Banco de dados	PostgreSQL + extensão PostGIS	Relacional para usuários e serviços, mais busca por raio com índice espacial
Documentação da API	OpenAPI 3 + Swagger UI (@fastify/swagger)	Documentação gerada do próprio código

Camada	Tecnologia	Por que
Testes	Vitest, testes de rota com o `inject` do Fastify, Playwright (opcional)	Testes rápidos no mesmo ecossistema
Qualidade	ESLint, Prettier, GitHub Actions	Padrão de código e testes a cada push
Infraestrutura	Docker e Docker Compose	Todo mundo sobe o ambiente igual com `docker compose up`

Hospedagem sugerida

Parte	Opções
Front-end	Vercel, Netlify ou Cloudflare Pages
API	Render, Railway ou Fly.io
Banco	Supabase ou Neon (ambos oferecem PostGIS)

Os planos gratuitos desses serviços mudam com frequência; conferir limites e condições antes de decidir.

APIs externas consumidas

API	Para que serve	Cuidado
ViaCEP	Transformar CEP em endereço (RF10)	Guardar o resultado em cache para não repetir consultas
Nominatim (OpenStreetMap)	Transformar endereço em latitude e longitude no cadastro de serviços	Política de uso: no máximo 1 requisição por segundo e `User-Agent` identificando o projeto
IBGE Localidades	Lista de estados e municípios para filtros	Dados quase estáticos: carregar uma vez e guardar
Tiles do OpenStreetMap	Imagens do mapa	Uso moderado; em produção real seria preciso um provedor próprio

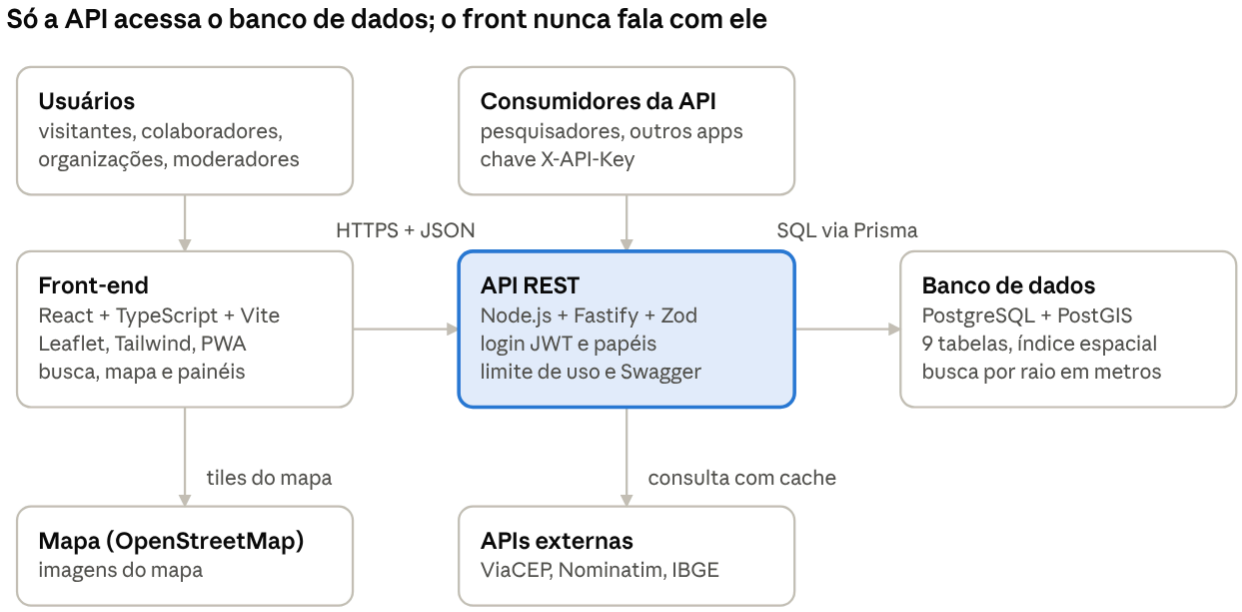
O endereço de cada API e as regras de uso devem ser conferidos na documentação oficial na hora de implementar; aqui estão descritos de memória, sem consulta online.

Por que não...

- **MongoDB:** os dados têm vínculos claros (usuário, serviço, verificação, denúncia) e pedem integridade referencial, que o modelo relacional dá de graça.
- **Google Maps:** exige conta de cobrança com cartão; Leaflet + OSM entrega o necessário sem custo.
- **Next.js full-stack:** reduz a separação front/back que a disciplina pede de forma explícita.

5. Arquitetura

O Pontelo tem três camadas (front-end, API e banco) em que o navegador conversa só com a API, e a API é a única que acessa o banco de dados e os serviços externos.



arquitetura do Pontelo · 3 camadas, 2 integrações externas

O front consome a API; a API consulta o banco e as APIs externas. O mapa é a única parte que o navegador busca diretamente fora da API.

Fluxo de uma busca

1. A pessoa informa onde está: GPS, CEP ou cidade. Com CEP, o front chama `GET /geocodificar`, e a API consulta o ViaCEP e o Nominatim, guardando o resultado em cache.
2. O front chama `GET /servicos` com `lat`, `lng`, `raio` e `categoria`.

3. A API valida os parâmetros (Zod), consulta o PostGIS com `ST_DWithin` e calcula o selo de confiança de cada serviço.
4. A API devolve JSON; o front mostra a lista e os marcadores. O TanStack Query guarda a resposta em cache e o service worker guarda a última busca para uso offline.

Fluxo de um novo serviço

1. Um colaborador logado envia `POST /servicos`; o serviço nasce como `pendente` e fica invisível ao público.
2. Um moderador abre a fila (`GET /moderacao/servicos`) e decide em `POST /moderacao/servicos/{id}/decisao`.
3. Aprovado, o serviço passa a `publicado` e aparece na busca, no mapa e na API pública.
4. Dali em diante, confirmações da comunidade renovam o selo de confiança; denúncias voltam para a fila de moderação.

Decisões de arquitetura

- **API sem estado:** o login por JWT dispensa sessão guardada no servidor, então a API pode rodar em mais de uma instância sem ajuste.
- **Cache de geocodificação em memória** (Map com prazo de validade) no MVP, para respeitar o limite do Nominatim; se o projeto crescer, troca-se por Redis.
- **Separação em camadas no back-end:** rota, controlador, serviço e repositório, para que as regras de negócio possam ser testadas sem subir o servidor.
- **Ambiente local único:** o Docker Compose sobe front, API e banco com PostGIS juntos.

6. Banco de dados

O banco tem nove tabelas em PostgreSQL; a que importa mais é `servicos`, que guarda a localização como ponto geográfico (PostGIS) para responder "o que existe perto de mim" com índice espacial.

Tabelas

Tabela	Campos principais	Função
`usuarios`	id (UUID), nome, email (único), senha_hash, papel, criado_em	Contas de colaboradores, organizações e moderadores
`organizacoes`	id, nome, cnpj (opcional), responsavel_id, verificada	ONGs e órgãos que mantêm serviços

Tabela	Campos principais	Função
`categorias`	id, slug, nome, icone, cor	Abrigo, refeitório, jurídico, saúde, CRAS, documentos etc.
`servicos`	id, nome, descricao, categoria_id, organizacao_id, endereco, cidade, uf, cep, localizacao, telefone, documentos_exigidos, status, ultima_verificacao_em, criado_por	O serviço em si
`horarios_funcionamento`	id, servico_id, dia_semana (0 a 6), abre, fecha	Permite o filtro "aberto agora" (vários intervalos por dia)
`verificacoes`	id, servico_id, usuario_id, tipo (confirmação ou moderação), resultado, comentario, criado_em	Histórico de quem confirmou ou revisou cada serviço
`denuncias`	id, servico_id, usuario_id (opcional), motivo, descricao, status, resolvido_por, criado_em	Reporte de dado errado ou serviço fechado
`chaves_api`	id, usuario_id, nome, hash_da_chave, ativa, ultimo_uso, criado_em	Acesso de consumidores da API pública
`refresh_tokens`	id, usuario_id, token_hash, expira_em, revogado	Renovação de sessão

Relacionamentos

- Um usuário cria vários serviços, verificações e denúncias.
- Uma organização tem vários serviços; um serviço pode não ter organização (cadastro de colaborador).
- Uma categoria agrupa vários serviços.
- Um serviço tem vários horários, várias verificações e várias denúncias.
- Um usuário tem várias chaves de API e vários refresh tokens.

Ciclo de vida de um serviço (`status`)

1. `pendente` : sugerido por colaborador ou organização, ainda invisível ao público.
2. `publicado` : aprovado por um moderador, aparece no mapa e na API.
3. `rejeitado` : moderador recusou, com comentário em `verificacoes` .
4. `arquivado` : serviço encerrou ou foi denunciado e confirmado como fechado.

Selo de confiança (regra proposta)

Cada serviço mostra uma cor segundo `ultima_verificacao_em` : verde ("verificado") se a última confirmação tem até 90 dias, âmbar ("a confirmar") se for mais antiga. O prazo de 90 dias é uma sugestão e pode mudar.

Exemplo de criação da tabela principal

```
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TYPE status_servico AS ENUM ('pendente', 'publicado', 'rejeitado',
'arquivado');

CREATE TABLE servicos (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  nome text NOT NULL,
  descricao text,
  categoria_id int NOT NULL REFERENCES categorias(id),
  organizacao_id uuid REFERENCES organizacoes(id),
  endereco text NOT NULL,
  cidade text NOT NULL,
  uf char(2) NOT NULL,
  cep char(8),
  localizacao geography(Point, 4326) NOT NULL,
  telefone text,
  documentos_exigidos text,
  status status_servico NOT NULL DEFAULT 'pendente',
  ultima_verificacao_em timestamptz,
  criado_por uuid NOT NULL REFERENCES usuarios(id),
  criado_em timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX servicos_localizacao_gix ON servicos USING GIST (localizacao);
CREATE INDEX servicos_categoria_status_idx ON servicos (categoria_id,
status);
```

Exemplo da consulta principal: serviços por perto

```
SELECT s.id, s.nome, c.slug AS categoria,
       ST_Distance(s.localizacao,
         ST_SetSRID(ST_MakePoint(:lng, :lat), 4326)::geography) AS
distancia_m
FROM servicos s
JOIN categorias c ON c.id = s.categoria_id
WHERE s.status = 'publicado'
  AND c.slug = :categoria
  AND ST_DWithin(s.localizacao,
        ST_SetSRID(ST_MakePoint(:lng, :lat), 4326)::geography, :raio_m)
ORDER BY distancia_m
LIMIT 20;
```

Atenção: o PostGIS recebe longitude primeiro e latitude depois em `ST_MakePoint`; trocar a ordem é o erro mais comum. O raio `:raio_m` é em metros.

7. API REST

A API segue o padrão REST com JSON, fica sob o prefixo `/api/v1` e a documentação interativa (Swagger UI) é servida em `/docs`. Leitura é pública; escrita exige login; a moderação exige papel de moderador.

Convenções

- **Autenticação de usuário:** cabeçalho `Authorization: Bearer <token>`.
- **Autenticação de consumidor da API:** cabeçalho `X-API-Key: <chave>`, com limite de requisições maior que o de visitantes anônimos.
- **Paginação:** parâmetros `pagina` e `limite` (máximo 50).
- **Nomes:** rotas e campos em português, `snake_case` nos campos JSON.
- **Limite de uso:** resposta `429` quando o IP ou a chave passa do limite.

Endpoints

Método	Rota	Quem acessa	O que faz
GET	`/servicos`	Público	Lista serviços; filtros: `lat`, `lng`, `raio` (m), `categoria`, `aberto_agora`, `q`

Método	Rota	Quem acessa	O que faz
GET	`/servicos/{id}`	Público	Detalhes de um serviço, com horários e selo de confiança
GET	`/categorias`	Público	Lista as categorias
GET	`/geocodificar`	Público	Converte `cep` ou `q` (endereço) em latitude e longitude, com cache
POST	`/auth/registro`	Público	Cria conta
POST	`/auth/login`	Público	Devolve token de acesso e refresh token
POST	`/auth/refresh`	Público	Troca o refresh token por um novo token de acesso
POST	`/auth/logout`	Logado	Revoga o refresh token
GET	`/me`	Logado	Dados do usuário atual
POST	`/servicos`	Logado	Sugere um serviço (nasce como `pendente`)
PATCH	`/servicos/{id}`	Dono, organização ou moderador	Edita um serviço
POST	`/servicos/{id}/confirmacoes`	Logado	"Confirmo que ainda funciona"; atualiza a última verificação
POST	`/servicos/{id}/denuncias`	Público (com limite de uso)	Reporta informação errada ou serviço fechado
GET	`/moderacao/servicos`	Moderador	Fila de serviços pendentes
POST	`/moderacao/servicos/{id}/decisao`	Moderador	Aprova ou rejeita, com comentário

Método	Rota	Quem acessa	O que faz
GET	`/moderacao/denuncias`	Moderador	Lista denúncias abertas
PATCH	`/moderacao/denuncias/{id}`	Moderador	Atualiza o status de uma denúncia
POST	`/chaves`	Logado	Gera uma chave de API (mostrada uma única vez)
DELETE	`/chaves/{id}`	Dono da chave	Revoga a chave
GET	`/dados/servicos.geojson`	Chave de API	Exporta serviços publicados em GeoJSON, para reuso por terceiros

Erros

Todo erro devolve o mesmo formato, o que facilita o tratamento no front:

```
{
  "erro": {
    "codigo": "VALIDACAO_FALHOU",
    "mensagem": "O campo lat deve estar entre -90 e 90.",
    "detalhes": [{ "campo": "lat", "problema": "fora_do_intervalo" }]
  }
}
```

Status	Quando
400	Requisição malformada
401	Sem login ou token inválido
403	Logado, mas sem permissão para a ação
404	Recurso não existe (ou ainda não foi publicado)
409	Conflito, por exemplo e-mail já cadastrado
422	Dados válidos no formato, mas inválidos nas regras de negócio
429	Limite de requisições excedido

Exemplo: serviços de refeição num raio de 3 km

Requisição:

```
GET /api/v1/servicos?
lat=-23.5505&lng=-46.6333&raio=3000&categoria=refeitorio&aberto_agora=true
```

Resposta (resumida):

```
{
  "dados": [
    {
      "id": "0b5c2c0e-5d3a-4b1c-9d3e-5d7a1f6e2a10",
      "nome": "Refeitório Exemplo",
      "categoria": "refeitorio",
      "distancia_m": 850,
      "aberto_agora": true,
      "selo": "verificado",
      "ultima_verificacao_em": "2026-09-20T14:30:00-03:00",
      "localizacao": { "lat": -23.5560, "lng": -46.6390 }
    }
  ],
  "pagina": 1,
  "total": 1
}
```

Os dados do exemplo são fictícios, só para ilustrar o formato.

Regras de negócio na API

- Só serviços `publicado` aparecem em rotas públicas; os demais são vistos pelo autor e por moderadores.
- Uma confirmação por usuário a cada 24 horas por serviço, para evitar inflar o selo de confiança.
- Editar dados sensíveis de um serviço publicado (endereço, telefone) devolve o serviço para `pendente` quando o autor não é organização verificada ou moderador.
- Denúncias repetidas de IPs diferentes sobre o mesmo serviço sobem a prioridade na fila de moderação.

8. Segurança, privacidade e LGPD

O público do Pontelo é sensível: quem busca abrigo ou atendimento pode estar em risco, então o princípio do projeto é guardar o mínimo possível sobre quem pesquisa e proteger bem quem cadastra.

Medidas de segurança

Risco	Mitigação
Vazamento de senhas	Hash com Argon2id; nunca registrar senha em log
Roubo de sessão	Token de acesso de 15 minutos guardado em memória; refresh token de 7 dias em cookie `HttpOnly`, `Secure` e `SameSite`, com rotação a cada uso e armazenado só como hash
Injeção de SQL	Consultas parametrizadas (Prisma e `$queryRaw` com template, nunca concatenação de texto)
XSS	React escapa o conteúdo por padrão; descrições são texto puro, sem HTML; cabeçalhos de segurança com `@fastify/helmet`
CSRF e acesso indevido entre origens	CORS restrito ao domínio do front e cookies `SameSite`
Abuso e robôs	Limite de requisições (`@fastify/rate-limit`); captcha opcional no formulário de denúncia
Acesso a rotas de moderação	Verificação de papel em toda rota, com testes automatizados para cada permissão
Chaves de API	Guardar só o hash da chave; exibir a chave completa uma única vez
Segredos no código	Variáveis de ambiente; `.env` no `.gitignore`; arquivo `.env.example` sem valores reais
Dependências vulneráveis	`npm audit` na integração contínua e alertas do Dependabot
Informação falsa ou golpe	Todo serviço novo nasce `pendente`; só aparece após aprovação

Proteção de quem busca ajuda

- **Saída rápida:** botão fixo que fecha o site e abre uma página neutra, útil para quem pesquisa em local onde não pode ser visto.
- **Sem histórico no servidor:** a localização do usuário só é usada para calcular a consulta; não é gravada em banco, e os logs devem omitir `lat` e `lng`.

- **Endereço restrito:** serviços de acolhimento a vítimas de violência costumam ter endereço sigiloso. Acrescentar o campo `endereco_restrito` (booleano) na tabela `servicos`: quando verdadeiro, a API mostra só o telefone de contato e a região.
- **Contatos institucionais:** publicar apenas telefones e e-mails de organizações, nunca números pessoais de voluntários.

Pontos de atenção da LGPD

Tema	Decisão proposta
Dados pessoais coletados	Apenas nome e e-mail de quem cria conta; visitantes não precisam se identificar
Finalidade e base legal	Execução do serviço de cadastro colaborativo, com consentimento explícito no registro
Direitos do titular	Tela para excluir a conta e exportar os próprios dados
Transparência	Política de privacidade em linguagem simples, linkada no rodapé
Denúncias	Anônimas por padrão; se guardar IP para evitar abuso, guardar só o hash
Retenção	Excluir tokens expirados e denúncias resolvidas após um prazo definido pelo grupo

Este quadro é um ponto de partida técnico e não é parecer jurídico; vale confirmar os detalhes com o professor da disciplina ou com a assessoria jurídica da faculdade.

9. Acessibilidade e UX

O Pontelo é desenhado para o celular mais simples, com internet fraca e pessoas com pouca familiaridade com tecnologia: poucos toques até o resultado, linguagem direta e lista antes do mapa.

Princípios

- **Lista primeiro, mapa depois:** a lista de resultados carrega rápido e funciona com leitor de tela; o mapa (Leaflet) é carregado sob demanda.
- **Peso da página:** meta de até 200 KB transferidos na tela inicial, sem contar os tiles do mapa.

- **Linguagem simples:** frases curtas, sem sigla solta. Exemplo: "CRAS (Centro de Referência de Assistência Social)" na primeira menção.
- **Cor nunca sozinha:** o selo "verificado" usa ícone e texto além da cor.
- **Áreas de toque:** botões com pelo menos 44 × 44 px.
- **Ação direta:** em cada serviço, botões grandes "Ligar" (`tel:`) e "Como chegar" (link para app de mapas).
- **Teclado e leitor de tela:** foco visível, ordem lógica, rótulos `aria-label` nos ícones, texto alternativo e mapa com alternativa em lista.
- **Offline:** o service worker guarda a interface e as últimas buscas, para o usuário rever endereços sem internet.
- **Estados claros:** mensagens de carregamento, erro e "nenhum serviço encontrado" sempre com um próximo passo (ampliar o raio, tentar outra categoria).

Telas do sistema

Tela	Objetivo
Início e busca	Escolher onde está (localização, CEP ou cidade) e a categoria de ajuda
Resultados	Lista ordenada por distância, com alternância para o mapa e filtro "aberto agora"
Detalhe do serviço	Endereço, horários, documentos exigidos, selo de confiança, botões Ligar e Como chegar, link para denunciar
Cadastro e login	Criar conta e entrar
Sugerir serviço	Formulário curto com busca de endereço por CEP e marcador ajustável no mapa
Painel da organização	Editar serviços, horários e contatos
Fila de moderação	Revisar sugestões e denúncias
Chaves de API	Gerar e revogar chaves, com atalho para a documentação

Como validar

- Lighthouse (desempenho, acessibilidade e PWA) a cada entrega de sprint.
- Extensão axe DevTools para achar problemas de acessibilidade.
- Teste com leitor de tela: TalkBack no Android ou NVDA no Windows.

- Simulação de rede lenta e CPU limitada no DevTools do Chrome.
- Teste de uso com pelo menos três pessoas de fora do curso, pedindo que achem um serviço sem ajuda.

10. Repositório e padrões de código

O projeto vive em um único repositório (monorepo) com front, back e documentação em pastas separadas, e sobe inteiro com `docker compose up`.

Estrutura de pastas

```
pontelo/
├── frontend/
│   ├── src/
│   │   ├── componentes/
│   │   ├── paginas/
│   │   ├── servicos/        # chamadas à API
│   │   ├── hooks/
│   │   └── estilos/         # tokens da paleta
│   └── public/              # logo, ícones PWA
├── backend/
│   ├── src/
│   │   ├── rotas/
│   │   ├── controladores/
│   │   ├── servicos/        # regras de negócio
│   │   ├── repositorios/    # acesso ao banco
│   │   ├── esquemas/        # validações Zod
│   │   └── plugins/         # auth, rate limit, swagger
│   ├── prisma/              # schema e migrações
│   └── testes/
├── docs/                    # esta documentação, diagramas, logo
├── docker-compose.yml
├── .env.example
└── README.md
```

Padrões do time

Tema	Padrão
Branches	`main` (estável, o que será apresentado), `dev` (integração) e `feature/<nome>` por tarefa
Commits	Conventional Commits: `feat:`, `fix:`, `docs:`, `test:`, `refactor:`

Tema	Padrão
Revisão	Todo merge em `dev` passa por pull request revisado por outra pessoa
Formatação	ESLint e Prettier com a mesma configuração no front e no back
Arquitetura do back	Rota, controlador, serviço e repositório: regra de negócio nunca fica na rota
Migrações	Toda mudança no banco entra como migração versionada (Prisma Migrate)
Nomes	Código e tabelas em português, sem acento; mensagens ao usuário com acento
Tarefas	Quadro Kanban (GitHub Projects ou Trello) com uma coluna "Em revisão"

Variáveis de ambiente (`.env.example`)

Variável	Uso
`DATABASE_URL`	Conexão com o PostgreSQL
`JWT_SECRET`	Assinatura dos tokens de acesso
`REFRESH_TOKEN_SECRET`	Assinatura dos refresh tokens
`FRONTEND_URL`	Origem permitida no CORS
`NOMINATIM_USER_AGENT`	Identificação do projeto na API de geocodificação
`VITE_API_URL`	Endereço da API usado pelo front

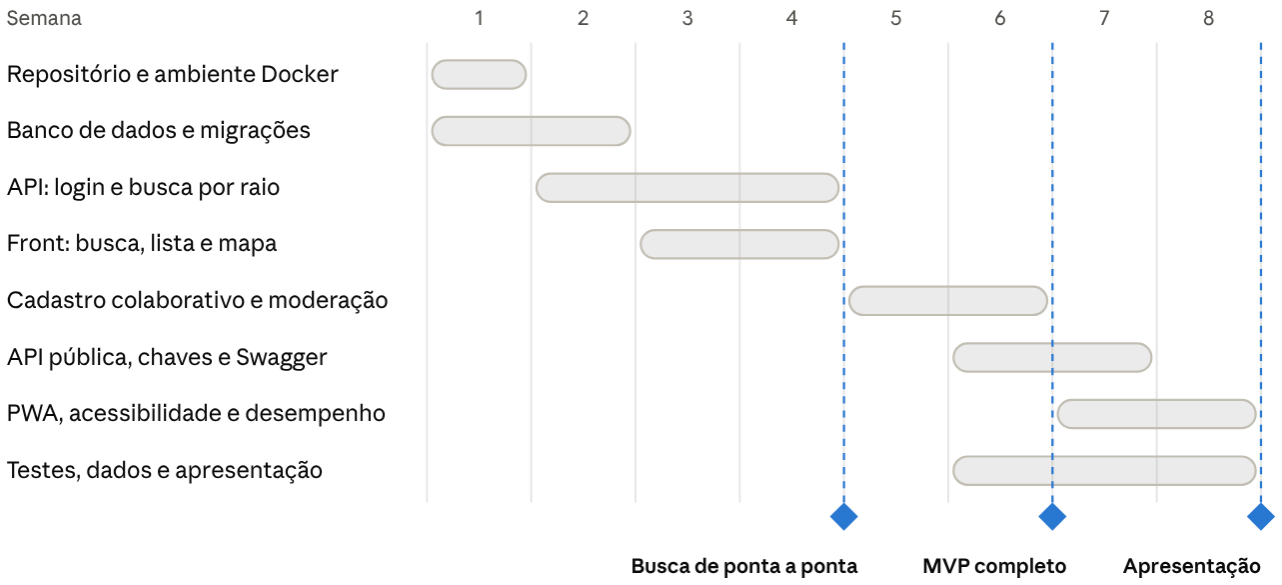
Para o README

O README do repositório deve ter, nesta ordem: o que é o Pontelo (com a logo), como rodar localmente em três comandos, como rodar os testes, link para a documentação da API e lista de integrantes.

11. Plano de desenvolvimento

O plano assume 8 semanas, porque o grupo não informou o prazo da disciplina; se o calendário for outro, a ordem das entregas continua valendo e só as semanas mudam.

Em 8 semanas, a busca fica pronta na semana 4 e o MVP na semana 6



cronograma de 8 semanas · 8 frentes, 3 marcos

Banco e API de busca abrem o caminho; o front entra na semana 3; cadastro, moderação e API pública fecham o MVP; as semanas finais cuidam de qualidade e apresentação.

Marcos de entrega

Marco	Semana	Critério para considerar cumprido
Busca de ponta a ponta	4	O usuário informa um CEP ou usa o GPS, vê serviços ordenados por distância na lista e no mapa, vindos do banco real
MVP completo	6	Colaborador sugere serviço, moderador aprova, serviço aparece na busca, confirmações e denúncias funcionam
Apresentação	8	Sistema publicado online, API documentada em `/docs`, testes passando e dados reais carregados

Divisão sugerida do trabalho

A divisão abaixo vale para um grupo de quatro pessoas; com menos gente, juntam-se as áreas vizinhas.

Área	Responsabilidades
Front-end	Telas, mapa, acessibilidade, PWA e consumo da API
Back-end	Rotas, autenticação, regras de negócio, moderação e chaves de API
Banco e dados	Modelagem, migrações, consultas geográficas e carga inicial de serviços
Qualidade e documentação	Testes, Swagger, README, integração contínua, identidade visual e roteiro da apresentação

Riscos e como reduzi-los

Risco	O que fazer
Mapa sem dados na hora da demonstração	Carga inicial a partir de dados abertos mais um núcleo de serviços conferidos à mão pelo grupo
PostGIS difícil de instalar em cada máquina	Usar a imagem Docker oficial do PostGIS no Compose
Limite de uso do Nominatim	Cache de geocodificação e consulta apenas no cadastro de serviços
Front e back andando em ritmos diferentes	Fechar o contrato OpenAPI na semana 1 e usar respostas simuladas no front até a API ficar pronta
Escopo grande demais	Cortar primeiro os itens "desejáveis" (RF11 a RF14), nunca a moderação nem a API
Ninguém para moderar durante a apresentação	Criar uma conta de moderador de demonstração e uma fila com serviços pendentes de exemplo

12. Fontes de dados e referências

Para o mapa não nascer vazio, a ideia é popular o banco com dados abertos e depois deixar a comunidade corrigir e completar. Nenhuma dessas fontes foi consultada durante a escrita deste documento; conferir disponibilidade, formato e licença de cada uma antes de importar.

Dados iniciais (seed)

Fonte	O que traz	Como usar
OpenStreetMap (Overpass API)	Locais marcados como instalação de assistência social (`amenity=social_facility`), com subtipos como abrigo, refeitório e banco de alimentos	Script que consulta a cidade escolhida e grava como `pendente` para moderação
CNES / DATASUS	Estabelecimentos de saúde cadastrados	Importar unidades de saúde e de vacinação
Dados abertos do Ministério do Desenvolvimento e Assistência Social	Equipamentos da assistência social, como CRAS	Importar endereços e contatos oficiais
Portais de dados abertos de prefeituras e estados	Listas locais de serviços	Complementar o que faltar na cidade da demonstração
Cadastro manual pelo grupo	Serviços que o grupo visitar ou ligar para confirmar	Garantir um núcleo pequeno de dados verificados para a apresentação

Todo dado importado entra como `pendente` e só é publicado depois de uma checagem humana, para não exibir informação desatualizada como se fosse confiável.

Contatos nacionais fixos (tela inicial)

A tela inicial pode exibir, sempre visíveis, os telefones nacionais de apoio, independentemente do mapa: SAMU 192, Disque 100 (direitos humanos), Ligue 180 (violência contra a mulher) e CVV 188 (apoio emocional). Confirmar os números e a disponibilidade nos sites oficiais antes de publicar.

Referências técnicas

Tema	Onde consultar
Mapa	leafletjs.com e openstreetmap.org
Geocodificação	nominatim.org (política de uso) e viacep.com.br
Municípios e estados	servicodados.ibge.gov.br (API de localidades)

Tema	Onde consultar
Banco geoespacial	postgis.net
Back-end	fastify.dev e prisma.io
Acessibilidade	w3.org/WAI/WCAG21 e webaim.org
Segurança	owasp.org (OWASP Top 10)
Privacidade	Lei Geral de Proteção de Dados (Lei nº 13.709/2018)

Os endereços acima foram escritos de memória; abrir cada um para confirmar que continua válido.